TableModel

GUI constructor : model = new TableModel{service.get--, this};
ui→tableView→setModel(model);

SELECTION (GUI)

```
QModelIndexList selection = ui→tableView→selectionModel()→selectedIndexes();
if (selection.empty()) return;
int row = selection.at(0).row();
auto stars = get....;
Star ~~star~~ selectedStar = stars[row];
```

no button

```
connect(ui→tableView→selectionModel(), &QItemSelectionModel::selectionChanged, this,
        &GUI::populateDiseasesList);
```

UPDATE IN CELL (Table Model)

```
Qt::ItemFlags BacteriaTableModel::flags(const ....) const {
    return Qt::ItemIsSelectable | Qt::ItemIsEnabled | Qt::ItemIsEditable;
}
```

Date in the past? `std::string currentDate = QDate::currentDate().toString("yyyy-MM-dd").toStdString();`

Sort by 2 things:

...save to file

```
std::string filename = ... + ".txt",
std::ofstream fout(filename);
fout << textEdit→toPlainText().toStdString();
fout.close();
...
```

find w matching text

```
for(...){
    if(s.getName().find(text)!=std::string::npos)
        result.push_back(s);
}
```

Parsing list

```
while(...){
    ....
    std::getline(ss, speciesString, ';');
    std::vector<std::string> species;
    std::string specie;
    std::stringstream sss(speciesString);
    while(std::getline(sss, specie, ';')){
        species.push_back(specie);
    }
}
```

Real time search

```
ui→listWidget→clear();
std::string text = ui→lineEdit→text().toStdString();
if(text.empty()) return;
auto question = service.get....
ui→listWidget→addItem(...);
auto answers = service.get....
for(...){
    ui→listWidget→addItem(...);
}
```

connect(ui→lineEdit, &QLineEdit::textChanged, this, ...)

auto item = new QListWidgetItem(QString::fromStdString...)

FONT

```
QFont font = item→font();
font.setBold(true);
item→setFont(font)
```

work

work

```
class BacteriaTableModel : public QAbstractTableModel{
    Q_OBJECT
private:
        std::vector<Bacterium> bacteriums,
public:
        explicit BacteriaTableModel(const ..... , QObject* parent = nullptr);
        int rowCount =
        int columnCount =
        QVariant data =
        void updateData(.... newData);
};
```

```
resizeColumnsToContents();
```

```
update(){
model->updateData(getCurrentDisplayed());
}
```

```
BacteriaTableModel::BacteriaTableModel(std::vec... ) : bacteriums{bacteriums},
    QAbstractTableModel{parent}{}
```

```
--row..... {
    return static_cast<int>(bacteriums.size());
}
--- column--- {
    return 4;
}
--- data--- {
if(!index.isValid() || role != Qt::DisplayRole)
        return QVariant{};
const auto& b= bacteriums[index.row()]
if(index.column() == 0)
    return QString::fromStdString(b.getName());
--- ---- -
if(.... ==3){
  QStringList diseaseslist;
  for(...d--){
     diseaseslist.append(QString::fromStdString(d));
  }
  return diseaseslist.join(",");
}
return QVariant{};
}
--- updateData ....{
beginResetModel();
bacteriums= newData;
endResetModel();
}
```

GUI.h

```
BacteriaTableModel* model;
```

.cpp

```
model = new BacteriaTableModel
{service.getBacte...(), this};
ui->tableView->setModel(mod
```

CheckBox

populate list ... →
```
std::vector<Patient> patients;
if (ui->checkBox->isChecked()) {
    patients = service.getSth....;
else
    patients = service.getSthElse---;
~~model->setStars(stars)~~ model->updateData(patients);
```

connectSignals ... →
```
connect(ui->CheckBox, &QCheckBox::toggled, ...)
```

QVBoxLayout ✓

```
void GUI::developIdea(){
    auto ideas = service.get...
    QWidget* newWindow = new QWidget{};
    auto layout = new QVBoxLayout{newWindow};
newWindow->setWindowTitle(----);
for(....: ideas){
    auto textEdit = new QTextEdit{};
    textEdit->setText(QString::fromStd.... (idea.getDescription()));
    auto saveButton = new QPushButton{"save"};
    layout->addWidget(textEdit);
    layout->addWidget(saveButton);
    connect(saveButton, &QPushButton::clicked, this, [this, textEdit, idea](){
        saveDevelopedIdea(---); });
}
window->setLayout(layout);
window->show();
}
```

Capture list (copy) ✓

SpinBox ✓

select
```
auto selection = ui->....
if (selection.empty()) {
    ui->spinBox->setEnabled(false);
    return;
}
....
ui->spinBox->blockSignals(true);
ui->spinBox->setValue(a.getVotes());
ui->spinBox->blockSignals(false);
ui->spinBox->setEnabled(a.getUserName() != user.getName());
....
```

connectSignals
```
connect(ui->spinBox, QOverload<int>::of(
  &QSpinBox::valueChanged), this, ...)
```

Update → service.updateVotes(a...., ui->spinBox->value());

Combo Box

constructor : ui → comboBox → addItem ("All")
for (....) {
    ui → comBOX → addItem (QString : from StdString(...));
}

connect Signals... connect (ui → comboBox, &QComboBox::currentTextChanged, th

function : std::string selected = ui → comboBOx → currentText().toStdString()
for (...) {
    if (selected != "All" && ... != selected)
        continue;
    ....
}

function QString category = ui → comboBox → currentText();
for (...) {
    if (category == "All" || ..... == category)
        result.push_back (...);
}

Slider

Qt → horizontalSlider → properties < min (or ... → setMinimum(
max (or ... → setMaximum(
value (10) .. (*)

connect (ui → slider, &QSlider::valueChanged, this, &GUI::radiusChang

(*)/ ui → horizontalSlider → setValue(10)
int val = ui → horizontalSlider → value();

↓

void ... (int val)
    radius = val;
    populateList()
}

QPainter window

```
public:
    explicit All (Service & service, ...);
    ~All() override;
    void update() override;
protected:
    void paintEvent(...) ...
private:
```

update() override; ← repaint();

paintEvent(...) ... ← QWidget::paintEvent(event);
QPainter painter(this);

Draw circle for each star in constellation:

```
QPainter painter(this);
auto stars = service.get.....
for(...){
  if(....)
    painter.setBrush(Qt::red);
  else
    painter.setBrush(Qt::black);
  int radius = s.getDiameter()/2;
  int x=s.getRA()+50; int y=s.getDec()+50;
  painter.drawEllipse(x-radius, y-radius, 2*radius, 2*radius)
}
```

Bacterium (species for each bact.)

```
QWidget::paintEvent(event);
QPainter painter(this);
std::map<std::string, int> speciesCount;
for(...b: bacteriums){
  speciesCount[b.getSpecies()]++;
int y=40;
for(... speciesCount){
    std::string species = pair.first;
    int count = pair.second;
painter.drawText(30,y, QString::fr...());
int x=30;
for(int i=0; i<count; i++){
    painter.drawEllipse(x, y+15, 20, 20);
    x+=30;
}
y+=40;
}
```

Circle for each Programmer

```
QPainter painter(this);
int startX=220;
int y=40;
for(const ..... programmers){
  int radius=... ;
  if(.....){
    painter.setBrush(Qt::blue);
    painter.setPen(Qt::blue);
  }
  else{
    painter.setBrush(Qt::NoBrush);
    painter.setPen(Qt::white);
  }
if(revised=0){
  painter.drawPoint(startX, y);
}
painter.drawText(20, y, QString::fra...);
painter.drawEllipse(startX-radius,
  y-radius, 2*radius, 2*radius);
y+=30;
}
```

STL ~~vectors~~

Search exact value: std:: find (v.begin(), v.end(), value);

ex: auto it = std:: find (v.begin(), v.end(), x);
if (it != v.end())
found;

search after a condition: std:: find (v.begin(), v.end(), condition);

ex: auto it = std:: find_if (v.begin(), v.end(), [](const Dog& d){
return d.getName() == "Rex";
});

count: std:: count (v.begin(), v.end(), value); → how many times does the value appear

ex: int cnt = std:: count (v.begin(), v.end(), 10);

count after a condition: std:: count_if (v.begin(), v.end(), condition) → how many elems respect the condition

ex: int c = std:: count_if (v.begin(), v.end(), [](const Source& s){
return s.getStatus == "revised";
});

sort:

ex: std:: sort (v.begin(), v.end(), [](const Package& p1, const Package& p2){
return p1.getStreet() < p2.getStreet();
});

~ filter for: std:: copy_if (v.begin(), v.end(), back_inserter(result), cond);

ex: std:: copy_if (packages.begin(), packages.end(),
std:: back_inserter(result), [](const Package& p){
return !p.getStatus();
});

move elems to delete at the end: std:: remove_if (v.begin(), v.end(), cond)

ex: v.erase (std:: remove_if (v.begin(), v.end(), [](const Package& p){
return p.getStatus() == true;
}), v.end());

∃ at least one? std:: any_of (v.begin(), v.end(), [](const Bacterium& b){
return b.getName() == "E coli";
});

all respect cond? ex: bool allDelivered = std:: all_of (packages.begin(),
packages.end(), [](const Package& p){ return p.status(); });

no one respects cond: std:: none_of (v..., v..., cond);

sums ← Accumulate: ex: int sum = std:: accumulate (scores.begin(), scores.end(), 0);